Module 9: Caching Information for Scalability

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 1: Introduction

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module objectives

This module prepares you to do the following:

- Explain when caching is used
- Describe caching with Amazon ElastiCache
- Describe caching with Amazon CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module overview

Sections

1. Introduction
2. Caching overview
3. Caching with ElastiCache
4. Caching with CloudFront
5. Caching strategies

Labs

- Caching Application Data with ElastiCache
- Implementing CloudFront for Caching and Application Security

Knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

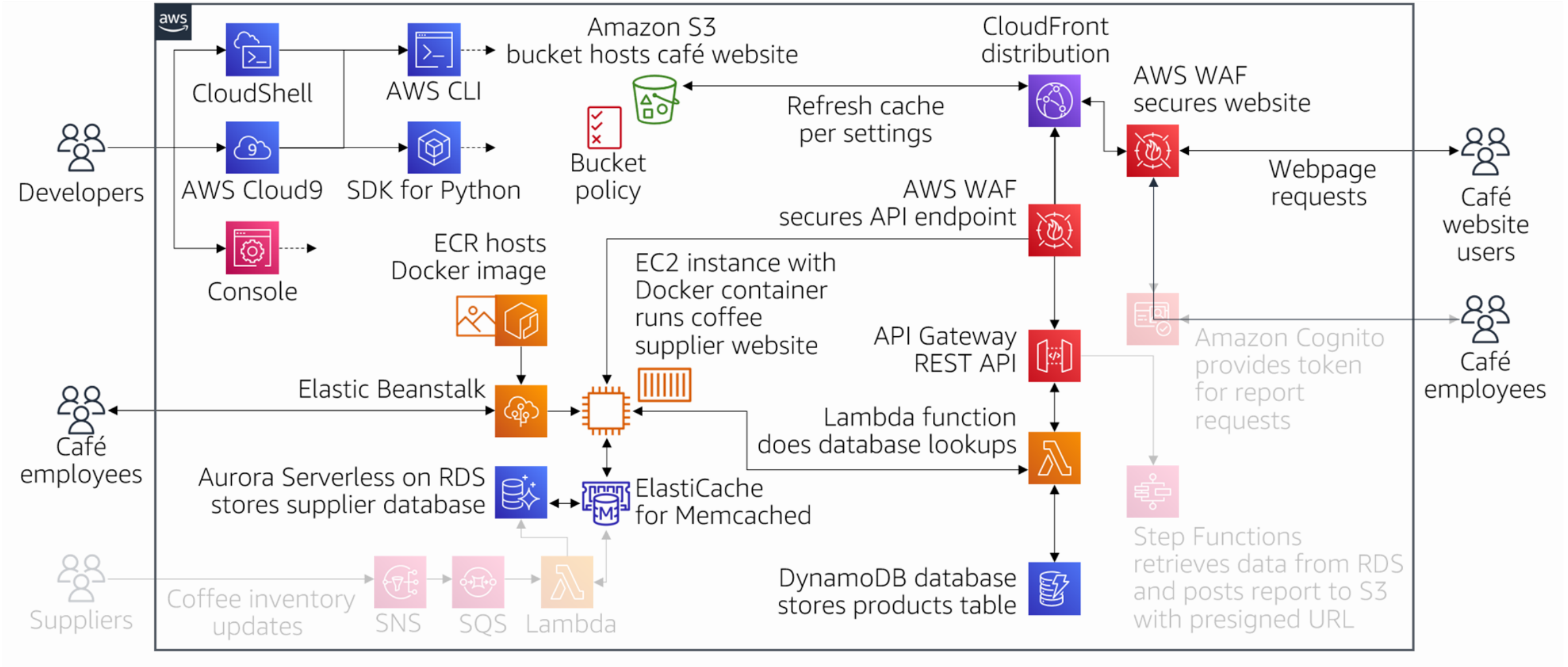
Café business requirement

The café employees have gotten occasional customer feedback that the coffee bean inventory information takes too long to display on the café website. Sofía wants to reduce the latency and also update the site to support HTTPS traffic.

```
cursor.close()

sql = "INSERT INTO hive (dev_id, humidity, temperature) VALUES (%s, %s, %s)"

def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))

    client.subscribe("+/devices/+/up")

def on_message(client, userdata, msg):
    print(msg.topic + " &&& " + str(msg.payload)+"\n")
    data = json.loads(msg.payload.decode('utf8').replace("'", '"'))
    val = (data["dev_id"], data["payload_fields"]["humidity"], data["payload_fields"]["temperature"])
    connection = mysql.connector.connect(
        host="bees-mariaDB",
        user="bees",
        passwd=mysql_password,
        database="bees"
    )
    cursor = connection.cursor()
    cursor.execute(sql, val)
    connection.commit()
    print(mycursor.rowcount, "record inserted.")
    cursor.close()
    connection.close()

client = mqtt.Client()
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Caching as part of developing a cloud application



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2: Caching overview

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Challenges of modern application data

Users

- Millions
- Globally distributed

Access methods

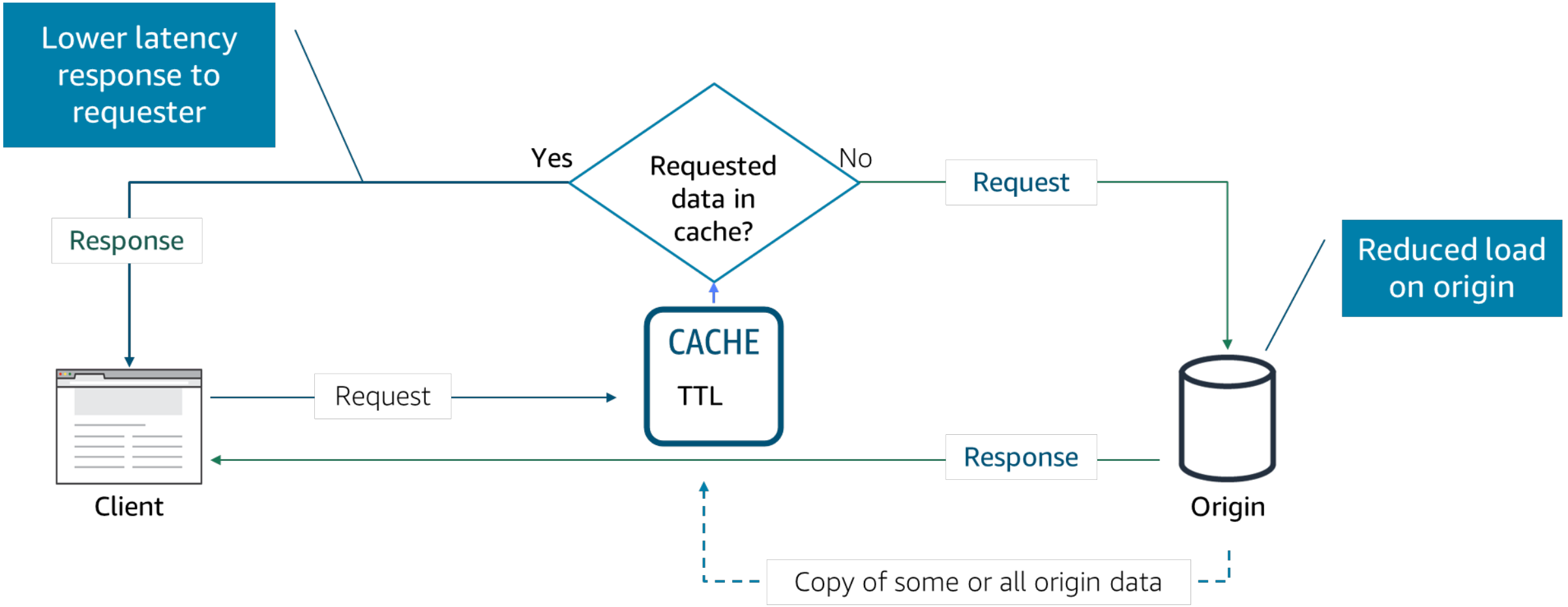
- Mobile
- Devices
- Internet of Things (IoT)

Requirements

- Millions of requests per second
- Microsecond latency

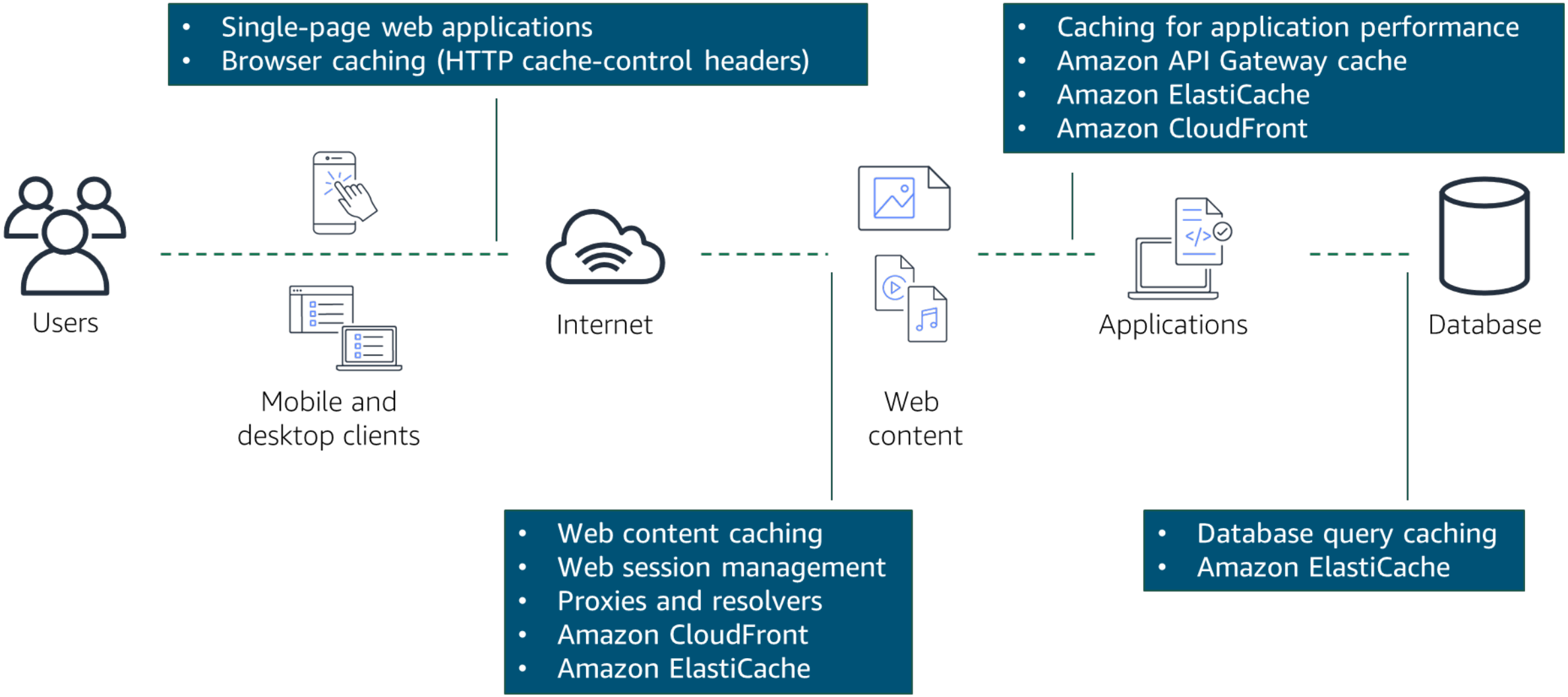
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What is caching?



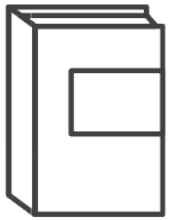
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

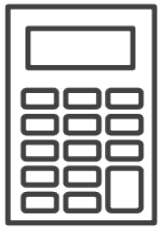
You can cache at all layers



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Workloads that benefit from caching





Read-Heavy Workloads	Compute-Intensive Workloads
Benefits • Reduce latency • Improve input/output operations per second (IOPS)	**Benefits** • Manipulate large datasets in real time
Examples • Social networking • Gaming • Media sharing • Question and answer (Q&A) portals	**Examples** • Recommendation engines • High performance computing (HPC) simulations

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Factors for considering whether to cache

Is a slow, expensive query required to generate the data?

Is the data relatively static and frequently accessed?

Can the data afford to be stale?

Will caching be worth the added development complexity and potential costs?

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2 key takeaways

- Modern applications put high demands on the **performance of data sources** in an application.
- Caching can improve performance by serving requests from a **copy of data** (a cache) rather than making the request to the origin.
- Caching **can occur at all layers** of your application.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3: Caching with ElastiCache

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ElastiCache

Amazon ElastiCache

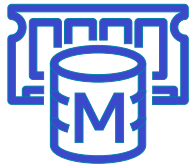
Fully managed **in-memory** data store with **sub-millisecond** latency

- Key-value, in-memory store that sits between your application and the origin data store
- Decreases access latency, increases throughput, and eases load of databases and applications
- Scales out, in, up, or down to meet demands
- Fully compatible with open-source Redis and Memcached engines

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ElastiCache engines



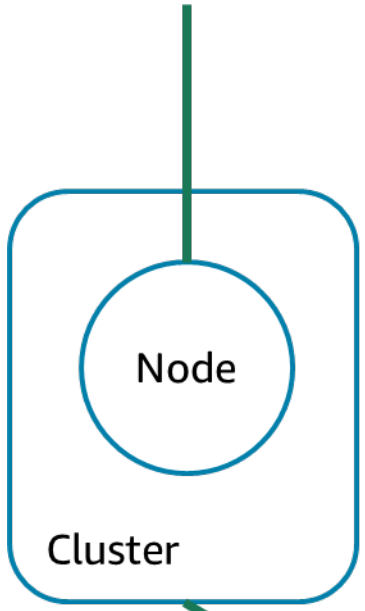
ElastiCache for Memcached	ElastiCache for Redis
Features • Multithreading • Low maintenance • Easy horizontal scalability with Auto Discovery	**Features** • Support for complex data types • Persistence • High availability (replication) • Automatic failover • Publish/subscribe capabilities
Suitability • Need the simplest model possible	**Suitability** • Need the advantage of data persistence • Need advanced data types • Need one of the other features that are listed

aws

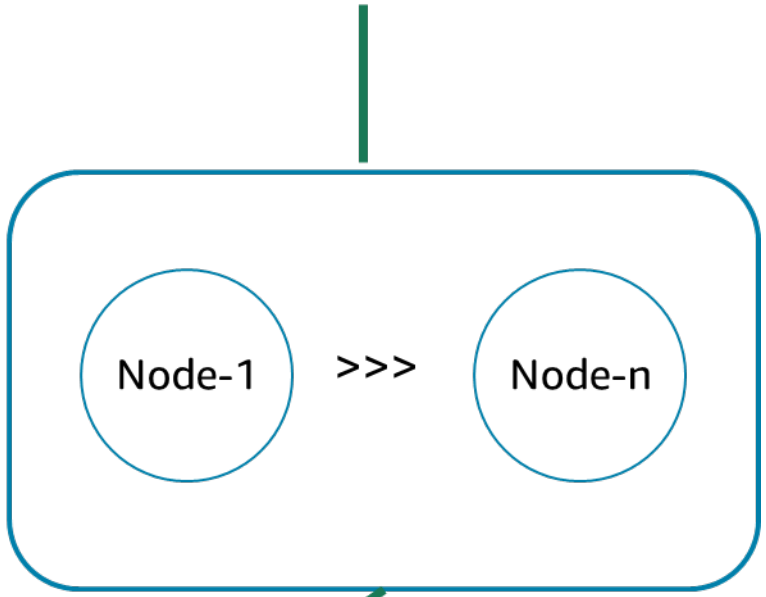
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ElastiCache terminology

Node: Fixed-size chunk of secure, network-attached RAM



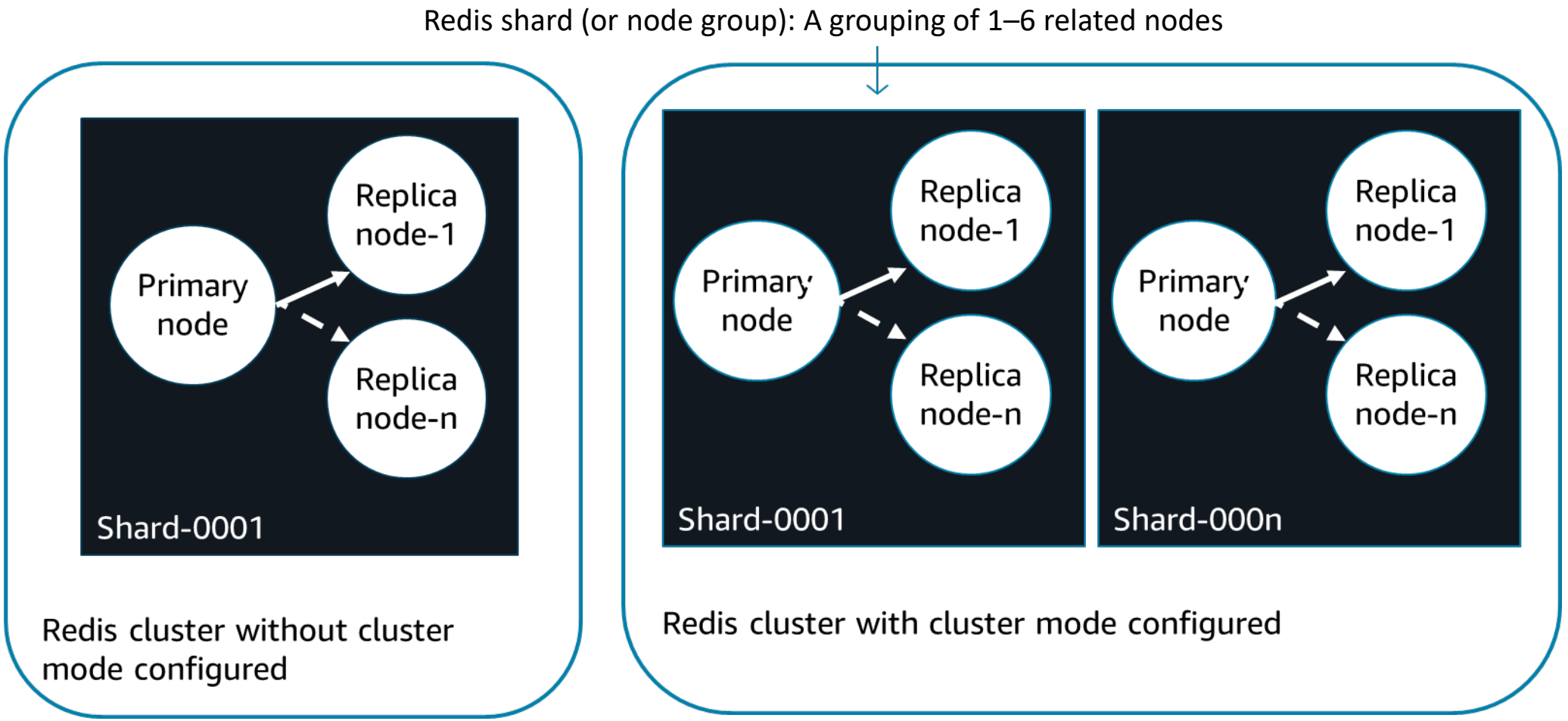
Memcached cluster with data partitioning



Cluster: Logical grouping of one or more ElastiCache nodes

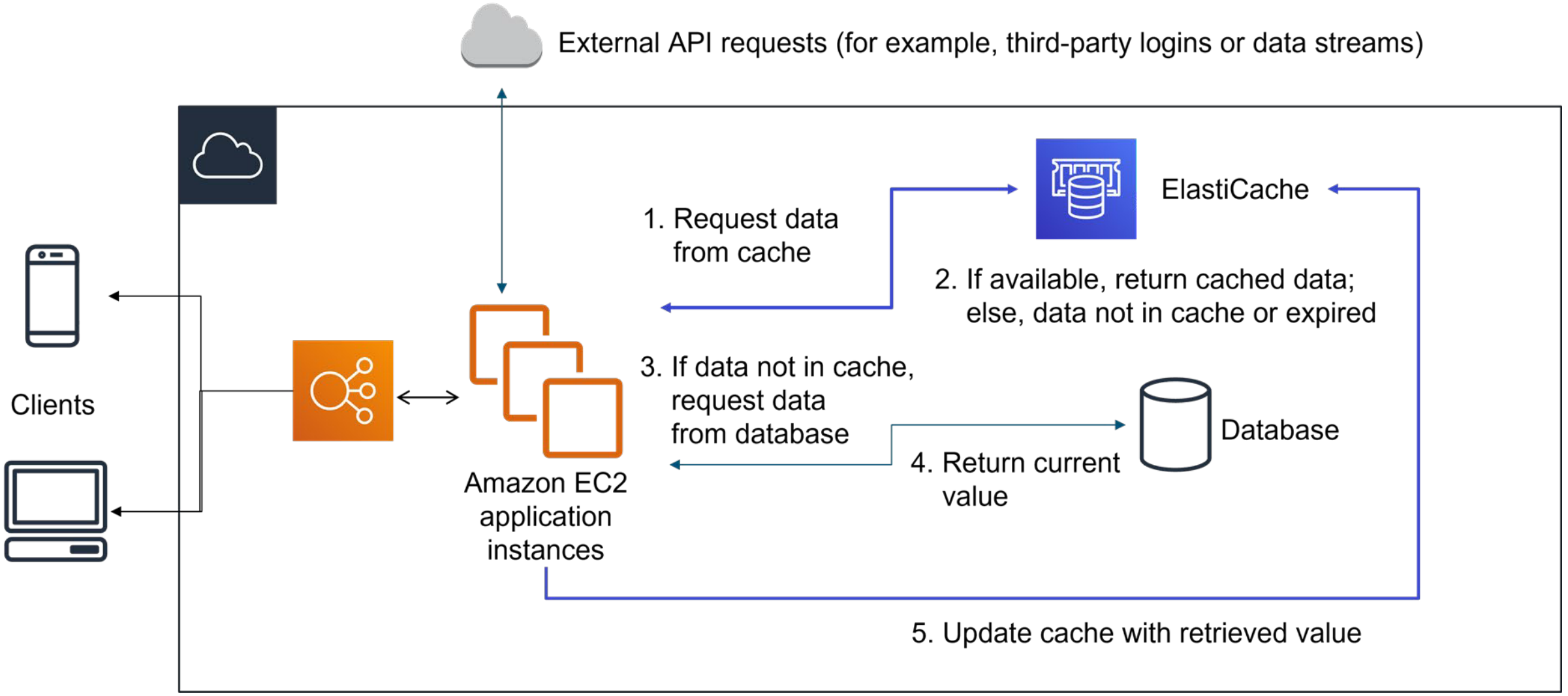
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ElastiCache terminology: Redis



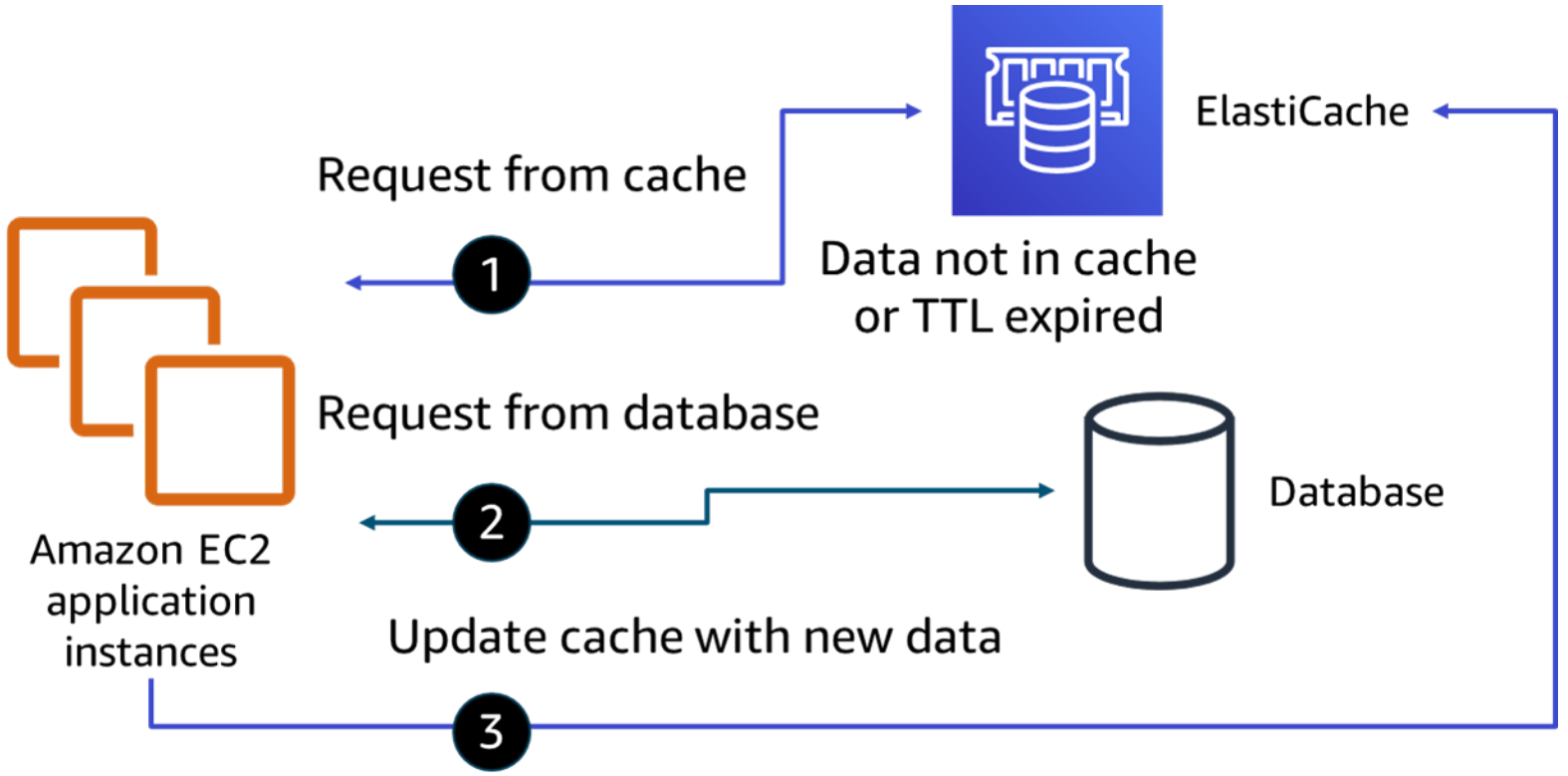
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: ElastiCache in an application



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Cache miss penalty and time to live (TTL)



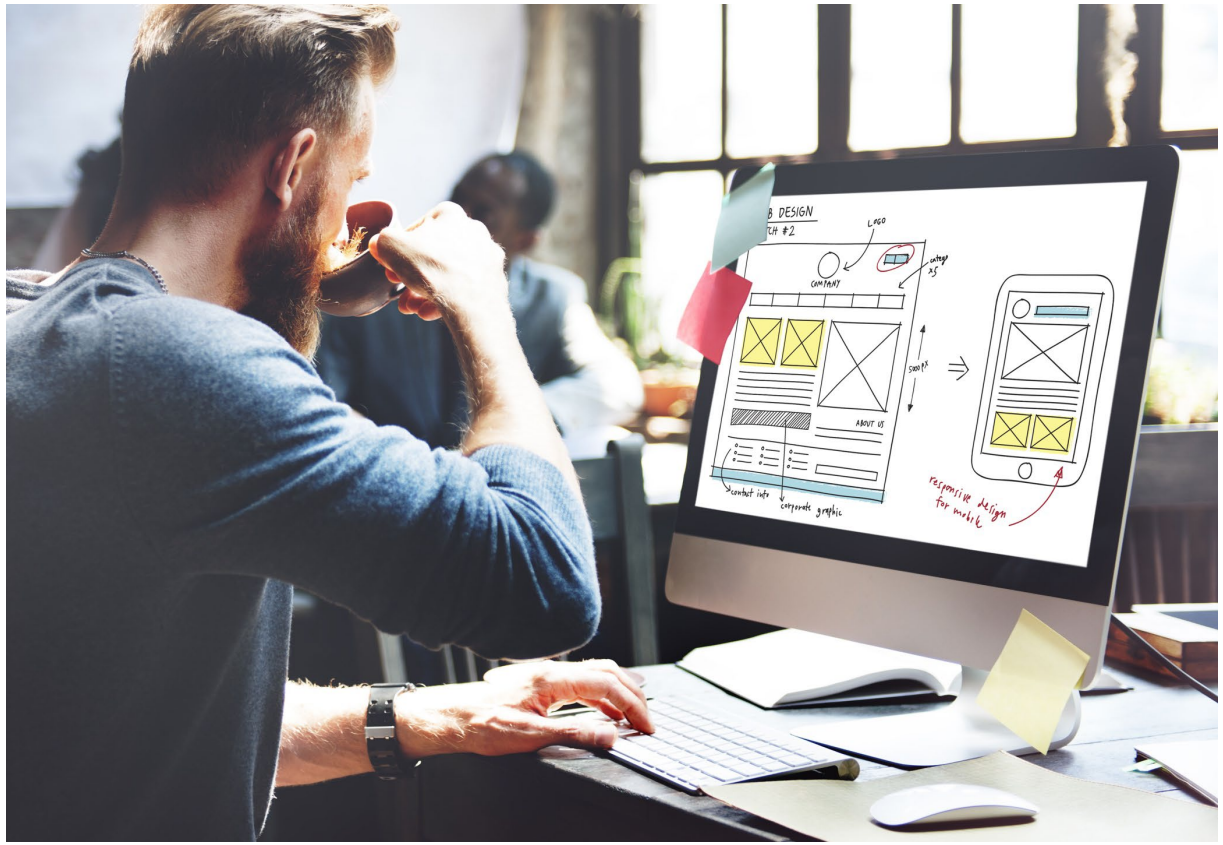
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3 key takeaways

- ElastiCache is a fully managed, **in-memory** data store with **sub-millisecond** latency.
- ElastiCache supports two open-source in-memory caching engines: **Memcached** and **Redis**.
- A **node** is a fixed-size chunk of secure, network-attached RAM, and a **cluster** is a logical grouping of one or more ElastiCache nodes.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 9.1: Caching Application Data with ElastiCache



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Scenario

- Frank and Martha are excited that the bean inventory from the coffee suppliers application is integrated into the café website. However, they are getting negative feedback that the bean inventory information takes too long to display on the page.
- One of the café's customers, Olivia, who is an AWS consultant, proposed adding database caching to improve the speed of the database query.
- Olivia recommended using the ElastiCache managed service. ElastiCache is similar to Amazon Aurora Serverless in that it simplifies the process to deploy and maintain a cache cluster.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

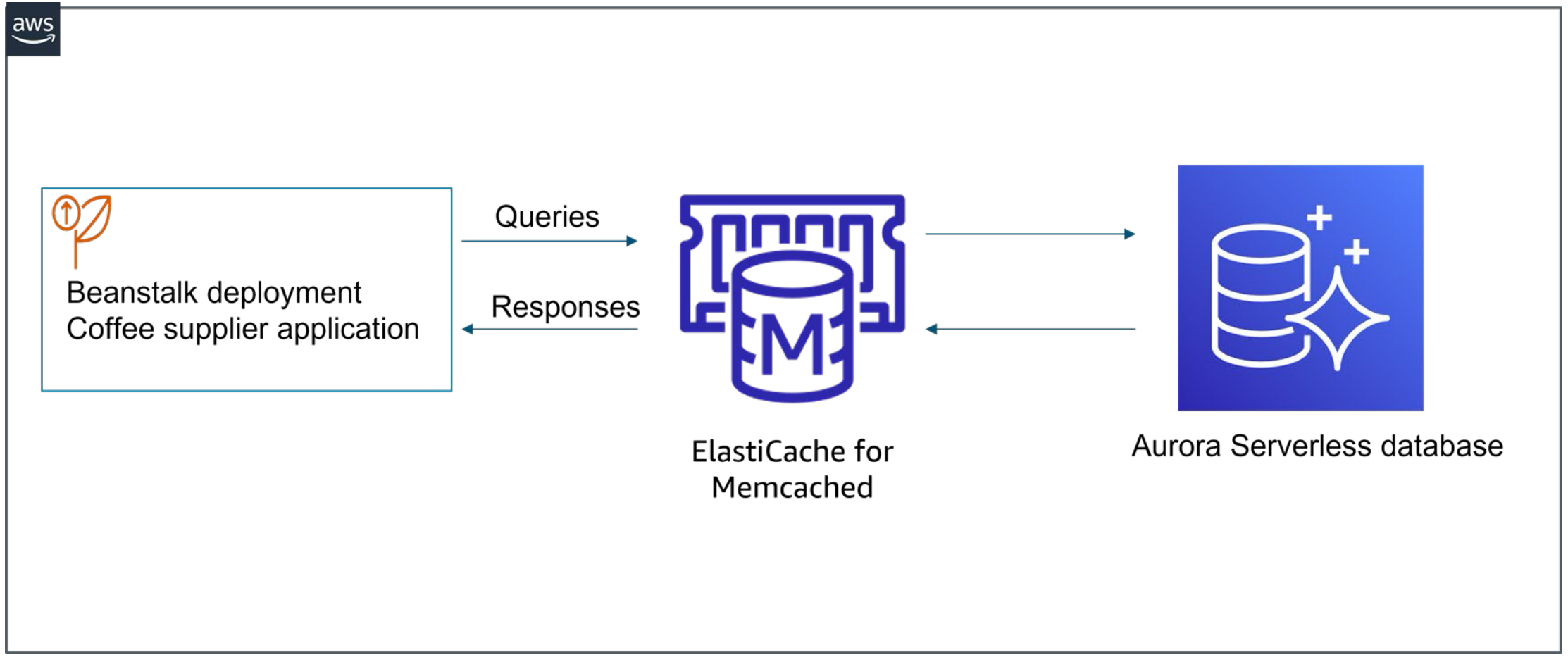
Lab: Tasks

1. Preparing the development environment
2. Configuring the subnets for ElastiCache to use
3. Creating the ElastiCache cluster
4. Loading records into the cache from the database
5. Updating data
6. Creating a new record
7. Deleting a record
8. Updating the application container

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 90 minutes

Begin Lab 9.1: Caching Application Data with ElastiCache

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

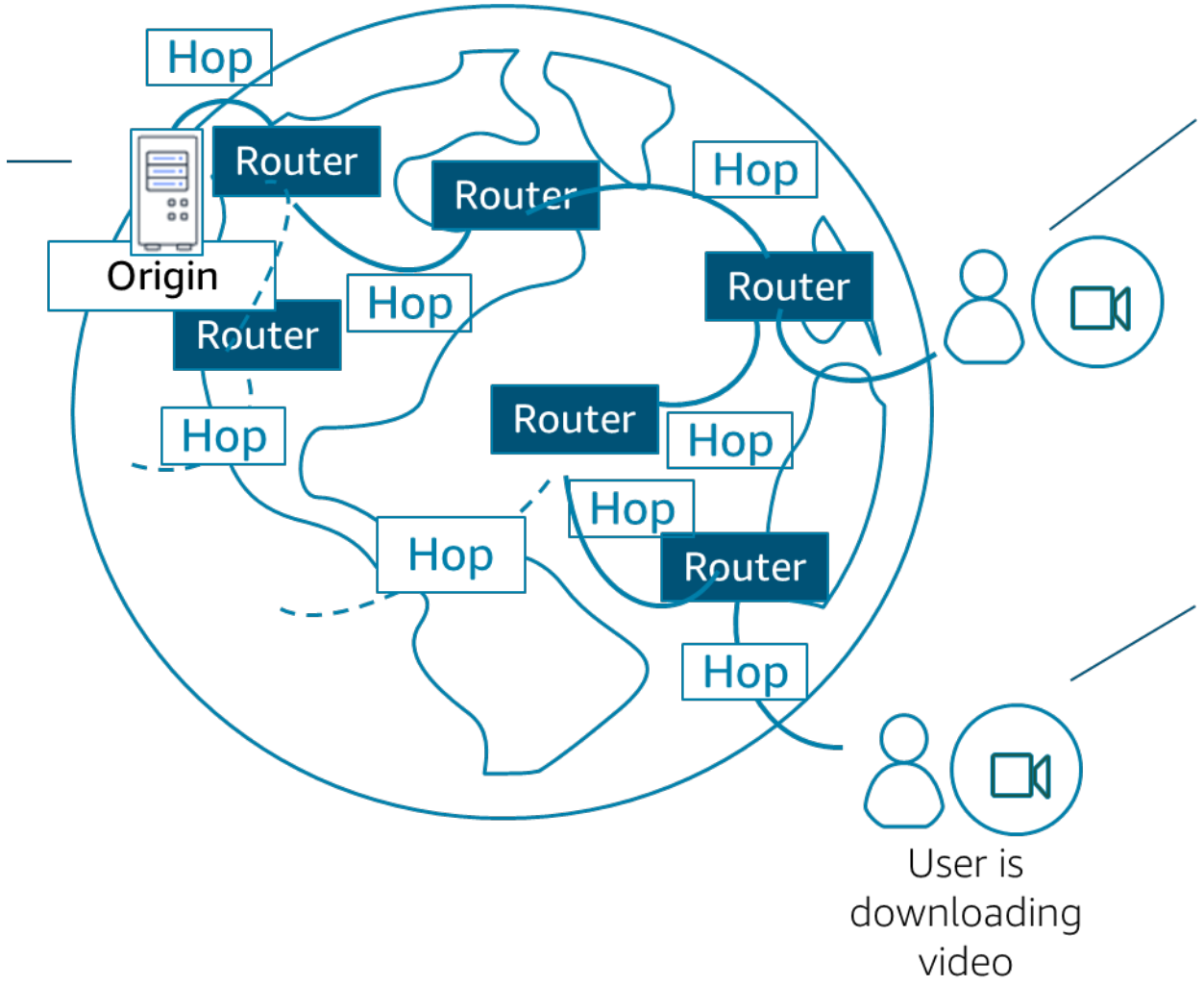
Section 4: Caching with CloudFront

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Challenges of serving data globally

High network latency between the origin and some users

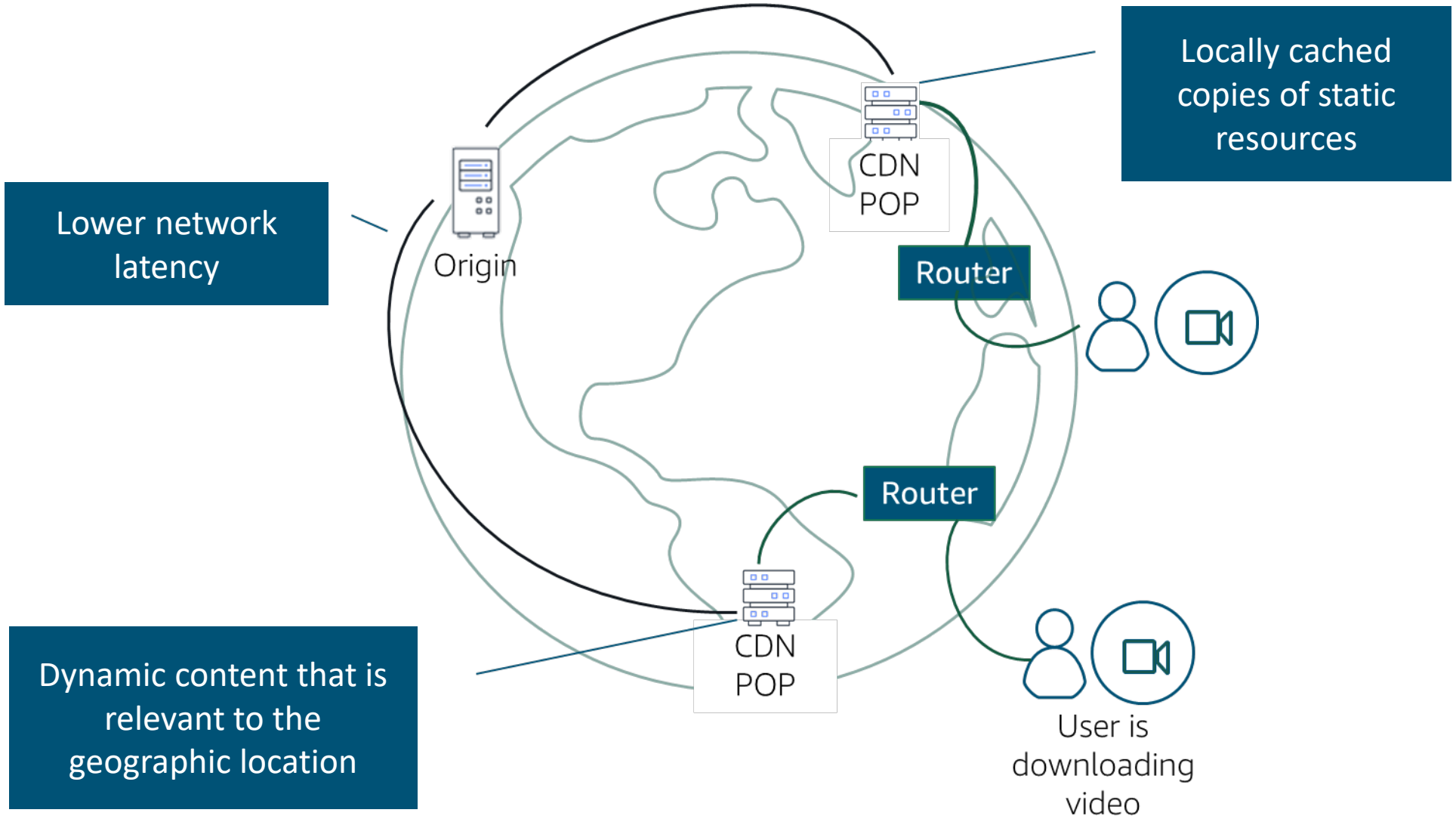


Greater distance between the origin and user affects performance

More hops between the origin and user affects performance

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What is a CDN?



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CloudFront

Amazon CloudFront

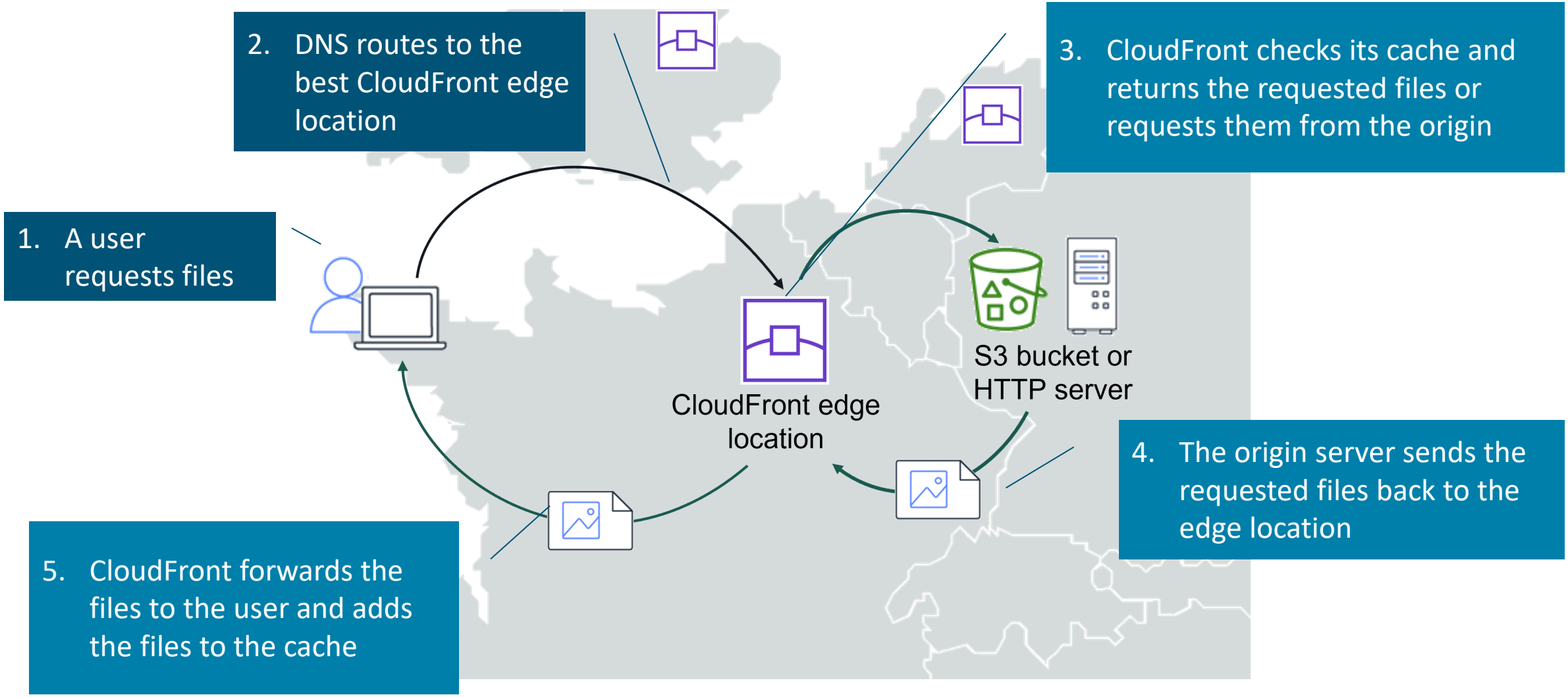
Fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers **globally** with **low latency** and **high transfer speeds**

- Optimized for performance and scale
- Built-in security features
- Deeply integrated with AWS services
- Highly customizable

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How CloudFront works



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CloudFront distribution types

Web content

- Used for serving content over HTTP or HTTPS, such as static and dynamic content.
- The origin can be an HTTP server or an S3 bucket.

Video on demand

- Used for video content that users can watch at any time.
- Videos must be encoded to support streaming.
- The origin can be an HTTP server or an S3 bucket.

Live event

- Used to stream live events.
- Videos must be encoded and compressed for viewing devices.
- The origin can be AWS Elemental MediaStore or AWS Elemental MediaPackage.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Understanding the cache key

The default cache key is the CloudFront distribution domain name plus the URL path of the requested object. Other values from the request are not included in the cache key by default. Examples follow.

Cache key:

- Domain: `d111111abcdef8.cloudfront.net`
- URL: `/content/stories/example-story.html`

Request 1: No matching cache key in cache. Cache miss

```
GET /content/stories/example-story.html?ref=0123abc&split-pages=false HTTP/1.1
Host: d111111abcdef8.cloudfront.net
User-Agent: Mozilla/5.0 Gecko/20100101 Firefox/68.0
Accept: text/html,*/*
Accept-Language: en-US,en
Cookie: session_id=01234abcd
Referer: https://news.example.com/
```

Request 2: Cache key matches request 1. Cache hit

```
GET /content/stories/example-story.html?ref=xyz987&split-pages=true HTTP/1.1
Host: d111111abcdef8.cloudfront.net
User-Agent: Mozilla/5.0 AppleWebKit/537.36 Chrome/83.0.4103.116
Accept: text/html,*/*
Accept-Language: en-US,en
Cookie: session_id=wxyz9876
Referer: https://rss.news.example.net/
```

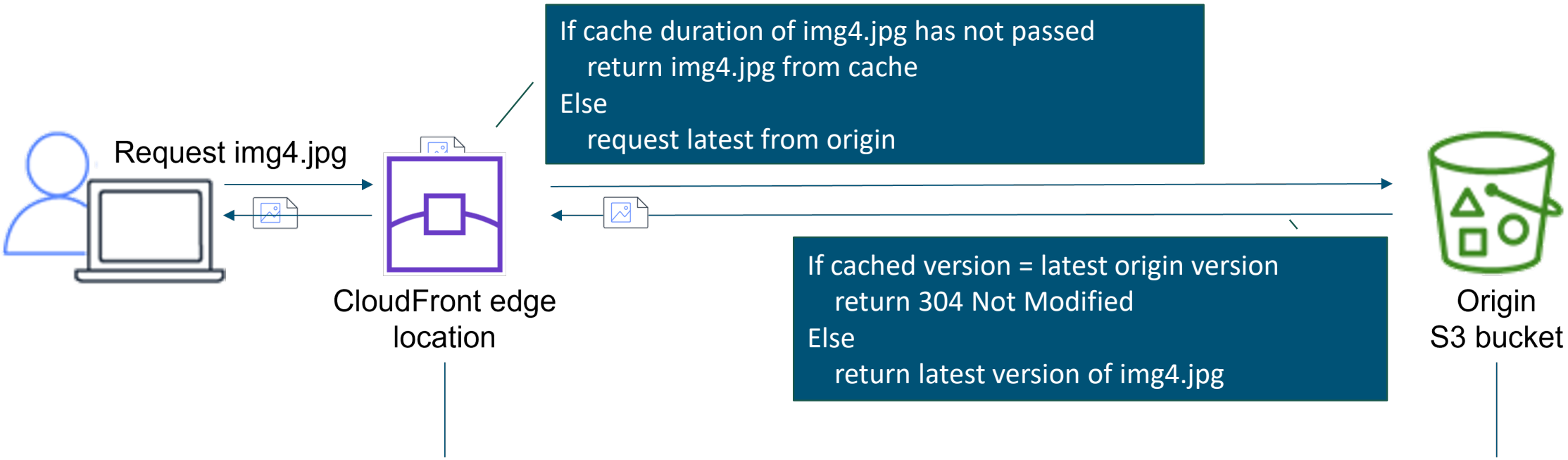
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Controlling cache keys with cache policies

Settings Category	Description	Available Settings
Policy information	Describes the policy.	• Name of policy (which is used to attach the policy to a caching behavior) • Comment
Time to live (TTL) settings	Work together with the `Cache-Control` and `Expires` HTTP headers (if they're in the origin response). They determine how long objects in the CloudFront cache remain valid before CloudFront checks the origin for updates.	• Minimum TTL • Maximum TTL • Default TTL
Cache key settings	Specify the values in viewer requests that CloudFront includes in the cache key. The values that you include in the cache key are automatically included in requests that CloudFront sends to the origin, which are called origin requests.	• URL query strings • HTTP headers • Cookies • Compression support

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Managing cached file expiration



Modify default duration (24 hours) with CloudFront TTL settings or origin response headers

In CloudFront

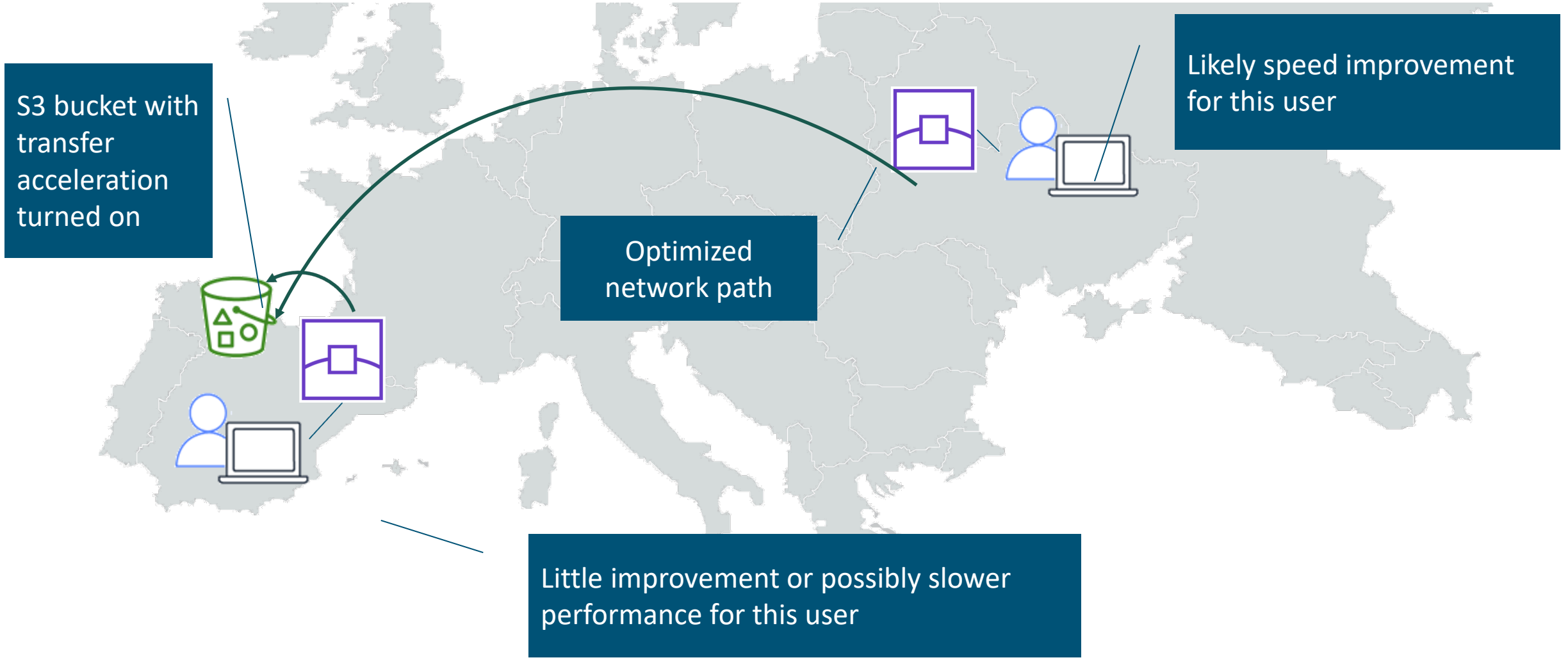
- Set Minimum TTL, Maximum TTL, and Default TTL in a cache policy
- Modifies duration for all files that match a path pattern

In the response header

- Include Cache-Control or Expires headers
- Modifies the cache duration of individual files

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using CloudFront to accelerate Amazon S3 file transfer



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

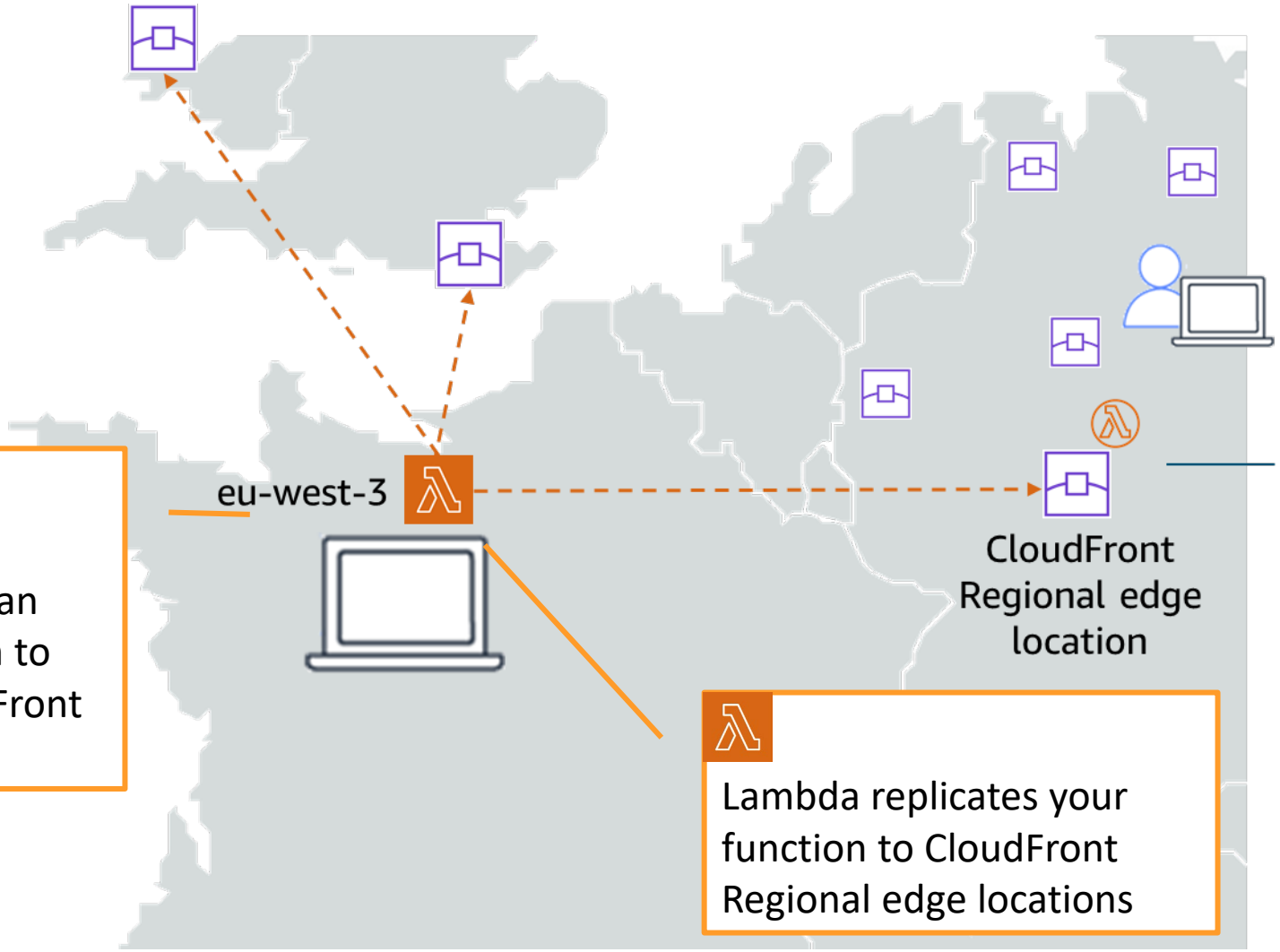
Customizing at the edge with functions

- Use edge functions to customize how your distribution processes requests and responses.
- Code runs close to viewers to minimize latency.
- Two ways to write and manage edge functions:
  - Lambda@Edge
  - CloudFront Functions

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using CloudFront with Lambda@Edge



Developer associates an AWS Lambda function to run based on a CloudFront distribution event

Lambda replicates your function to CloudFront Regional edge locations

A configured CloudFront event occurs, and the function is invoked at a CloudFront Regional edge cache

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

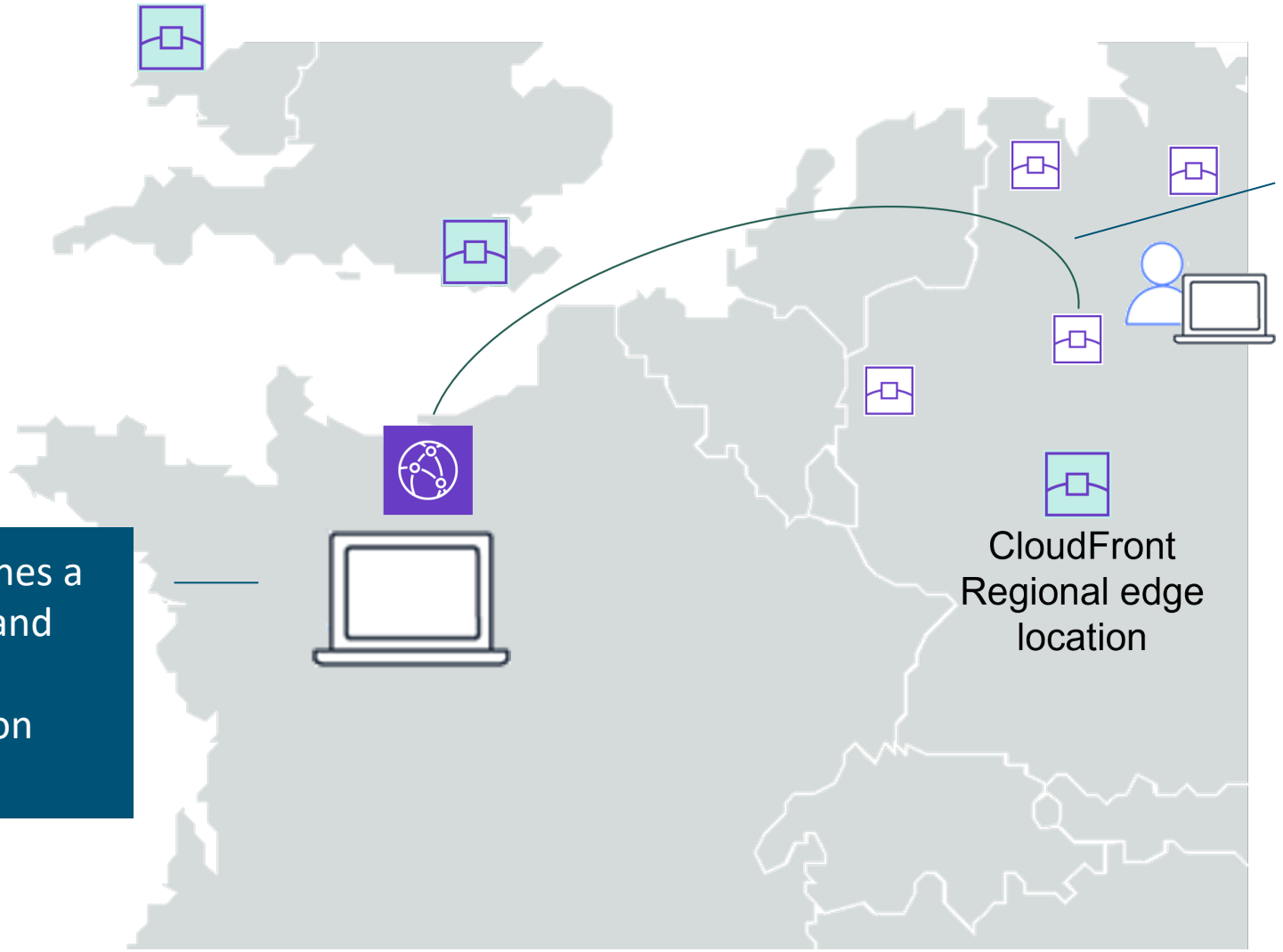
Lambda@Edge example

Lambda function code

```
def lambda_handler(event, context):
    request = event['Records'][0]['cf']['request']

    viewerCountry = request['headers'].get('cloudfront-viewer-country')
    if viewerCountry:
        countryCode = viewerCountry[0]['value']
        if countryCode == 'GB' or countryCode == 'DE' or countryCode == 'IE':
            domainName = 'eu.example.com'
            request['origin']['custom']['domainName'] = domainName
            request['headers']['host'] = [{'key': 'host', 'value': domainName}]
    return request
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using CloudFront Functions



The developer publishes a CloudFront function and associates it to a CloudFront distribution event

CloudFront intercepts requests and responses at edge locations and passes them to your function

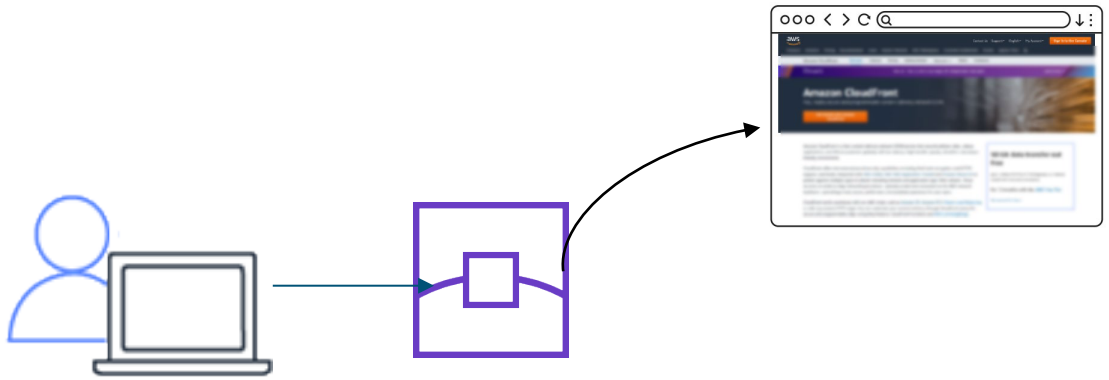
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CloudFront function example

CloudFront function code

```
function handler(event) {
    // NOTE: This example function is for
a viewer request event type.
    // Choose viewer request for event
type when you associate this function
with a distribution.
    var response = {
        statusCode: 302,
        statusDescription: 'Found',
        headers: {
            'cloudfront-functions': {
value: 'generated-by-CloudFront-
Functions' },
            'location': { value:
'https://aws.amazon.com/cloudfront/' }
        }
    };
    return response;
}
```

CloudFront function redirects to:
`https://aws.amazon.com/cloudfront/`



Requested URL (the CloudFront distribution):
`https://d111111abcdef8.cloudfront.net/`

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Choosing between Lambda@Edge and CloudFront Functions

	Lambda@Edge	CloudFront Functions
Use cases	Allows complex functions and full application logic	Lightweight, short-running functions
Ability to build and test entirely within CloudFront	No Extension of **Lambda**	Yes Native feature of CloudFront
Programming languages	Node.js or Python runtime	JavaScript
Event sources	Viewer request/response and origin request/response	Viewer request/response
Scale	Greater duration, memory capacity, and function size than CloudFront Functions	Limited to sub-milliseconds of duration, 2 MB memory, and 10 KB in size

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4 key takeaways

- CloudFront is a fast **CDN** service that delivers data globally with **low latency** and **high transfer speeds**.
- CloudFront distributions can serve **web**, **video**, or **live streaming** content.
- A **cache key** is the **unique identifier** for each object in the cache.
- You can use **CloudFront Functions** and **Lambda@Edge** to **customize** how your distribution processes requests and responses.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5: Caching strategies

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Two primary caching strategies

Lazy loading

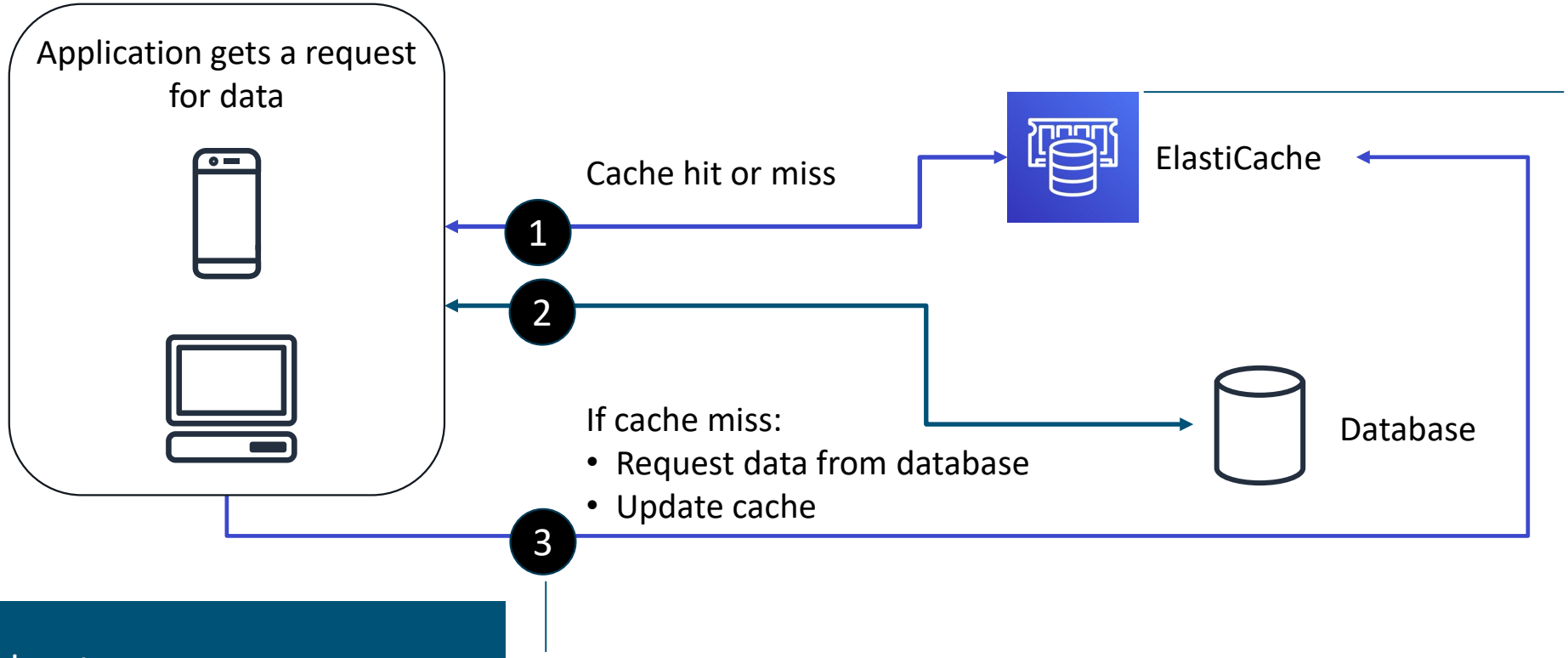
- The cache is updated only when necessary (cache hit or cache miss).
- Use when data will be read often but written infrequently.
- Example: User profile information

Write-through

- The cache is updated whenever data is written to the database.
- Use when data must be updated in real time.
- Examples: Top 100 game leaderboard or top 10 most popular news stories

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lazy loading



Advantages
- Less unneeded data in cache
- Non-fatal node failures

Disadvantages
- Cache miss penalty
- Potential for stale data

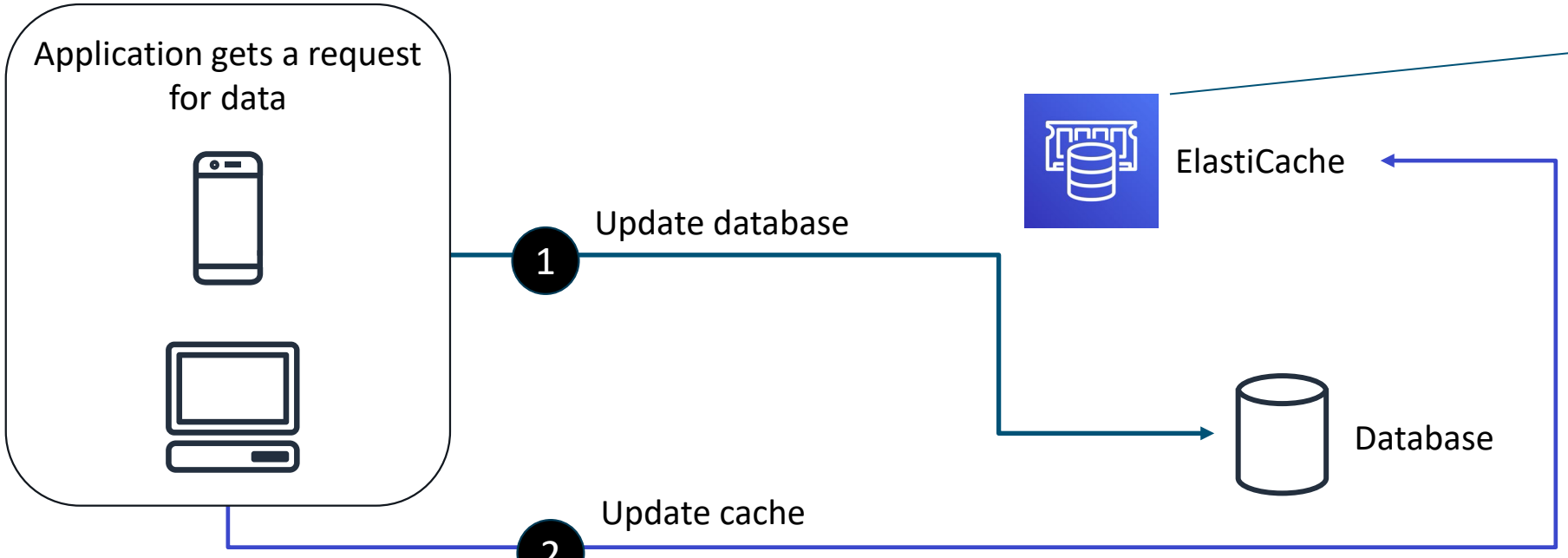
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lazy loading example

```
# Python
def get_user(user_id):
    # Check the cache
    record = cache.get(user_id)
    if record is None:
        # Run a DB query
        record = db.query("select * from users where id = ?", user_id)
        # Populate the cache
        cache.set(user_id, record)
    return record
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Write-through



Advantage
- No stale data in the cache

Disadvantages
- Write penalty
- Missing data on node failure
- Unused data in cache
- Cache churn

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Write-through example

```
# Python
def save_user(user_id, values):
# Save to DB
record = db.query("update users ... where id = ?", user_id, values)
# Push into cache
cache.set(user_id, record)
return record
```

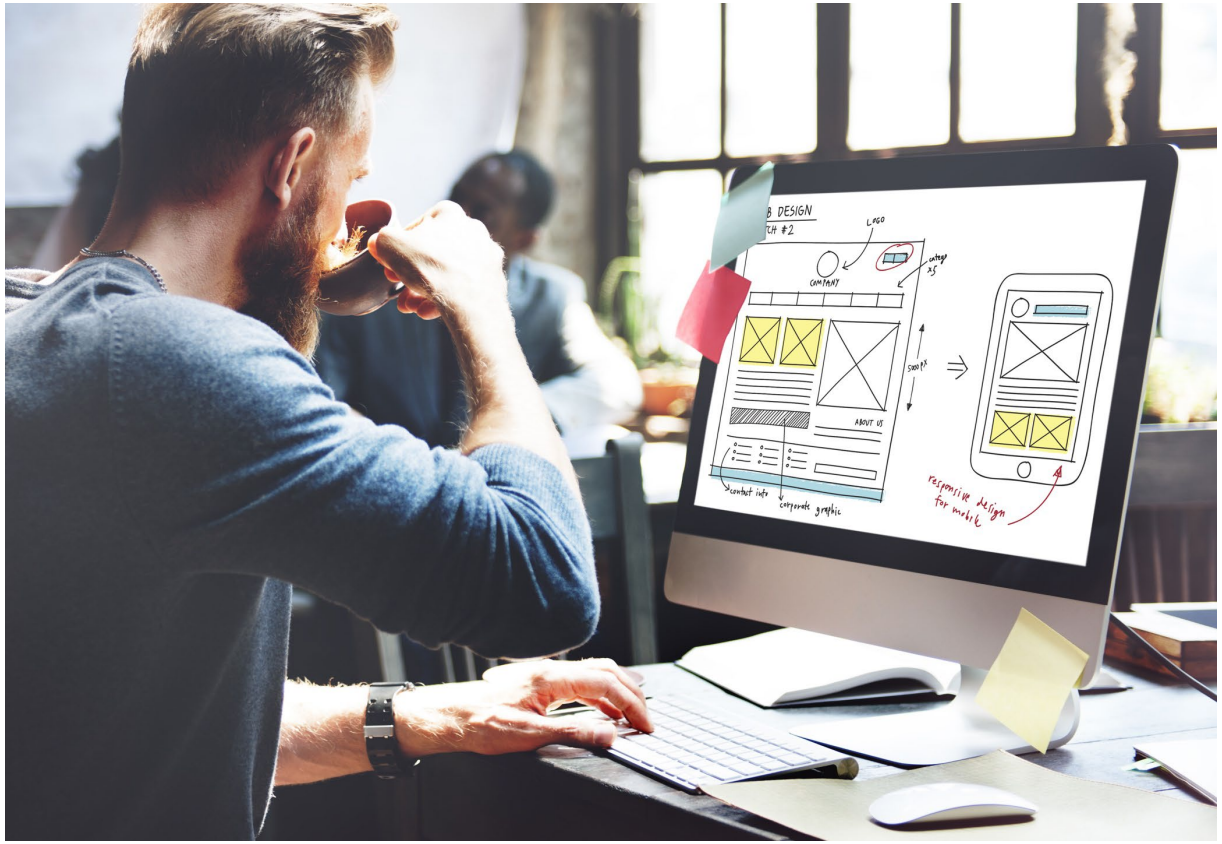
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5 key takeaways

- Caching has two primary strategies: **lazy loading** and **write-through**.
  - With **lazy loading**, the cache is updated **only when necessary** (cache hit or miss).
  - With **write-through**, the cache is updated **whenever data is written** to the database.
- Choose a caching strategy that **matches the use case** for your data.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 9.2: Implementing CloudFront for Caching and Application Security



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Scenario 2

- The café website is still running on HTTP and does not yet support HTTPS.
- The café would like to ensure webpages will load quickly for users globally.
- Sofía decides it is time deliver the café website through a **content delivery network (CDN)**.

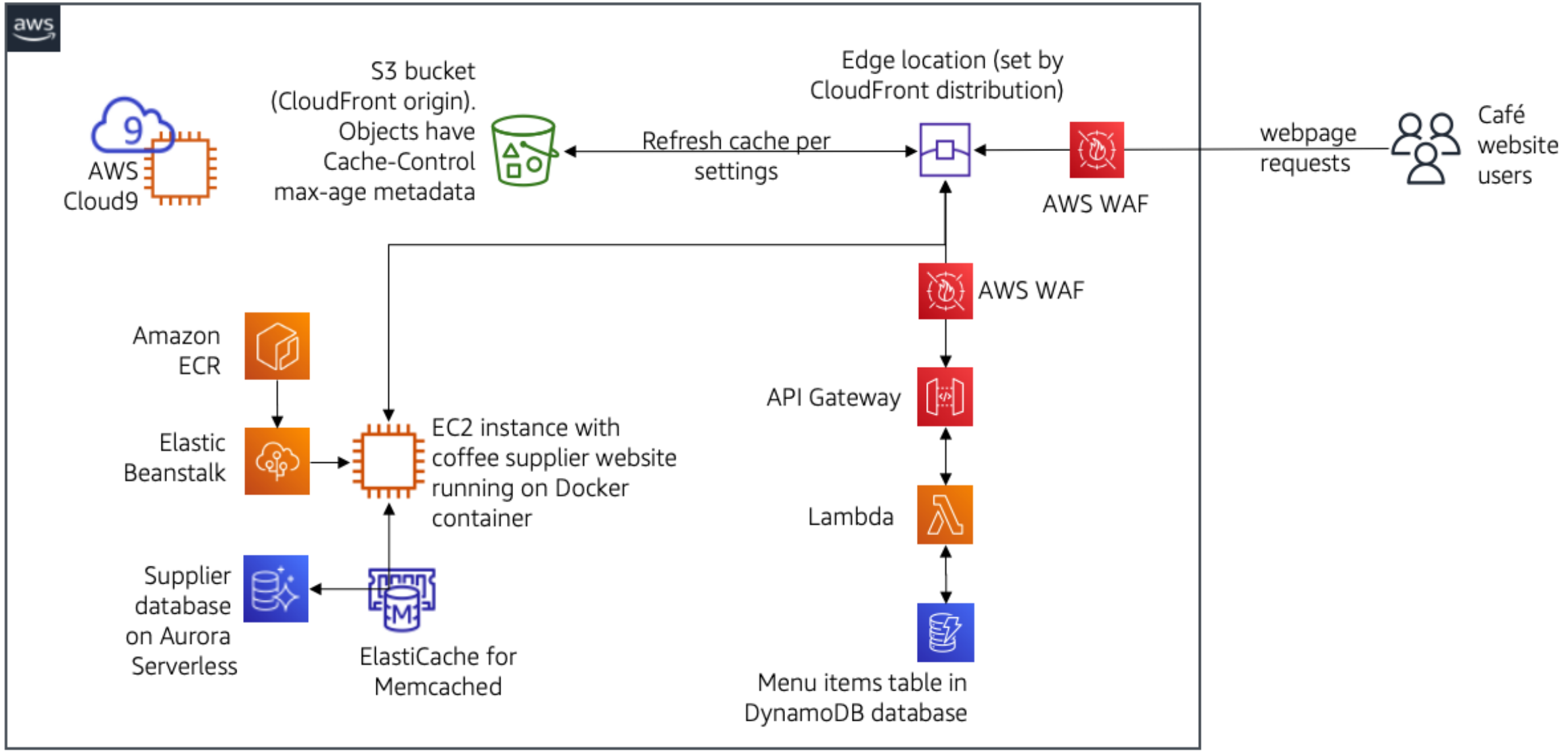
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Tasks 2

1. Preparing the development environment
2. Configuring an Amazon CloudFront distribution for static website content
3. Securing network access to the CloudFront distribution using AWS WAF
4. Securing a REST API endpoint using AWS WAF
5. Configuring a CloudFront Function on the website
6. Adjusting the cache duration

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 90 minutes

Begin Lab 9.2: Implementing CloudFront for Caching and Application Security

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module wrap-up

Module 9: Caching Information for Scalability

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module summary

In summary, in this module, you learned how to do the following:

- Explain when caching is used
- Describe caching with ElastiCache
- Describe caching with CloudFront
- Describe and compare Lambda@Edge and CloudFront Functions
- Apply caching strategies

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Complete the knowledge check

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question

Which requirement is a reason to choose Memcached over Redis for your ElastiCache configuration?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Requirement**
- **Memcached over Redis**
- **ElastiCache**

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Responses

Which **requirement** is a reason to choose **Memcached over Redis** for your **ElastiCache** configuration?

Choice	Response
A	You need data replication.
B	You need the simplest model possible.
C	You have complex data types.
D	You need an in-memory cache.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	You need data replication.
B	**You need the simplest model possible.**
C	You have complex data types.
D	You need an in-memory cache.

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Additional resources

Video:

- Getting Started with Caching: Turbocharging Your Application Workloads

Blog post:

- Introducing CloudFront Functions – Run Your Code at the Edge with Low Latency at Any Scale

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.